TRIPCANVAS

Technical Design Document (TDD)

Enterprise Travel Platform & Versioned Content Management System

Prepared For: IsoCode Workspace

Submission Track: Software Engineering Intern Take-Home Assignment

Client Case: Company A, Kochi, Kerala

Document Version: 1.0.0 (Production Release)

Date: November 2026

Repository & Live Application Links

GitHub Repository: **[INSERT PUBLIC GITHUB URL]**
Live Application: **[INSERT LIVE APPLICATION URL]**

* Replace bracketed placeholders upon repository publishing and Vercel domain assignment.

2. Executive Summary

TripCanvas is an enterprise-grade full-stack travel platform and versioned content management system (CMS) engineered for **Company A**, a luxury boutique tour operator based in Kochi, Kerala. The platform bridges the divide between aspirational travel editorial and transactional operations, enabling tour operators to market bespoke South India itineraries, manage seasonal pricing with strict financial integrity, ingest multi-channel traveller enquiries, and streamline consultation through server-side artificial intelligence.

The system is built upon modern web architecture utilizing Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS, Supabase PostgreSQL with Row Level Security (RLS), and Google's official Gemini Generative AI SDK (@google/genai). By prioritizing a bespoke editorial aesthetic over generic UI templates, TripCanvas delivers an immersive consumer experience alongside a rugged, high-throughput administrative console.

3. Problem Statement & Business Context

Company A faced substantial operational friction across its traditional workflow, hindering business expansion:

- **Developer Dependency for Catalog Updates:** Package itinerary modifications, hotel tier reclassifications, and seasonal tariff adjustments historically required engineering intervention, creating severe marketing lag.
- **Scattered Enquiry Management:** Customer enquiries arrived via fragmented, unmonitored channels (ad-hoc email, WhatsApp, phone calls) without centralized ticket tracking, leading to slow response times and missed booking conversions.
- **Manual and Inaccurate Quote Preparation:** Consultants manually referenced spreadsheets to calculate rates across changing peak and monsoon seasons. This manual process led to pricing discrepancies, unquoted GST, and confusion around child occupancy.
- **Absence of Version History:** Live edits directly mutated public pages, leaving no historical audit trail regarding what package terms or hotel categories were active when a traveller confirmed their reservation.

4. Project Objectives and Scope

The system scope is categorized into four verified boundaries:

- **Implemented Features:** Public package catalog (`/packages`) with destination, duration, price, and seasonal filtering; detailed package pages (`/packages/[slug]`) with day-wise itineraries, inclusions/exclusions, and transparent pricing tables; validated public enquiry forms with honeypot spam protection and rate limiting; versioned CMS editor with immutable publishing

and snapshot rollback; CRM enquiry inbox with SLA tracking and status transitions; server-side Google Gemini requirement extraction with confidence scoring; deterministic package recommendations strictly using database records; simulated editorial reply generation with mandatory consultant approval; full audit logging; and permanent 301 legacy URL redirect management with bulk CSV import.

- **Partially Implemented Features:** Supabase remote database integration: complete SQL migrations, schema constraints, and RLS policies are authored in `supabase/migrations/` and `supabase/seed.sql`, with an active fallback in-memory persistent store to allow immediate, zero-friction local execution without prior cloud migration.
- **Planned Features (Roadmap):** Interactive branded PDF quote generator, multi-currency live exchange rate conversion, bi-directional WhatsApp Business API synchronization via webhooks, and automated consultant workload balancing.
- **Explicitly Excluded Features:** Live payment gateway execution (e.g. Razorpay/Stripe checkout) and automated outbound message delivery to live telephone carriers, deliberately omitted to prevent unauthorized financial commitments or spam during sandbox evaluation.

5. Functional Requirements

- **FR-01 (Catalog Discovery):** Search published packages by destination keyword, filter by duration tiers (1-4d, 5-7d, 8d+), starting price ranges, and active seasons, and sort results ascending or descending.
- **FR-02 (Transparent Price Calculation):** Display verified per-person pricing, twin-sharing basis disclosures, statutory 5% GST disclosures, and explicit notices that child pricing requires confirmation.
- **FR-03 (Enquiry Ingestion & Ack):** Public travellers can submit travel requirements without creating accounts; valid submissions create persistent records with unique reference IDs (e.g. `ENQ-2026-001`) and generate immediate system acknowledgement logs.
- **FR-04 (Versioned CMS Management):** Staff can create draft packages, preview unpublished changes without altering live pages, and execute transactional publishing that updates package pointers and preserves prior versions.
- **FR-05 (Non-Destructive Rollback):** Restoring an earlier version creates a brand-new version snapshot from historical data without mutating or deleting prior versions.
- **FR-06 (AI Parameter Extraction):** Server-side Gemini analyzes traveller messages to extract destination, month, duration, adults, children, budget, and desired experiences with confidence ratings.
- **FR-07 (Deterministic Recommendation):** Matches packages against extracted parameters using a deterministic 100-point algorithm, strictly referencing published database packages.
- **FR-08 (AI Reply Drafting & Approval):** Drafts an editorial email response populated strictly with database pricing records; requires manual consultant approval before marking as dispatched.

- **FR-09 (Audit Activity Logging):** All administrative actions (creation, editing, publishing, rollback, assignment, and approval) generate immutable audit records.
- **FR-10 (Legacy URL Redirects):** Maps outdated website URLs to modern canonical package routes via 301 permanent redirects, with single-entry and bulk CSV import capabilities.

6. Non-Functional Requirements

- **Performance:** Sub-2.5 second initial page render on 4G networks; verified via Next.js static page prerendering, automatic code splitting, and optimized Unsplash remote image caching.
- **Security:** Zero client exposure of secrets (Gemini API key and Supabase service keys are server-side only); PostgreSQL Row Level Security; HTTPOnly SameSite cookie session management; Zod schema validation; and honeypot bot defense.
- **Reliability:** Graceful degradation: if Google Gemini is unreachable or credentials are unconfigured, the system falls back to rule-based defaults, flags `review_needed: true`, and never drops or blocks an enquiry.
- **Maintainability:** Full TypeScript strict mode coverage across all modules; zero build warnings; modular separation across `lib/`, `components/`, and `app/` routes; 100% automated test passage.
- **Scalability:** Stateless server route handlers and transactional database operations designed for deployment on edge or serverless cloud infrastructure (Vercel/Supabase).
- **Usability & Accessibility:** Responsive across mobile, tablet, and desktop viewports; semantic HTML elements; accessible form labels; and high-contrast typography conforming to WCAG AA guidelines.

7. Technology Stack

Technology	Version	Purpose	Where Used
Next.js (App Router)	15.5.28	Full-stack framework, SSR, routing	Entire application (`src/app/`)
React	19.0.0	Component architecture & reactivity	Frontend UI and forms
TypeScript	5.7.3	Static type checking & schema safety	Entire project codebase
Tailwind CSS	3.4.17	Custom luxury Kerala design system	Styling & responsive layout
Supabase / PostgreSQL	2.49.1	Database persistence & RLS	`supabase/` and `src/lib/supabase/`
Google Gen AI SDK	0.2.0	Server-side LLM extraction & drafting	`src/lib/ai/gemini.ts`
Zod	3.24.2	Schema validation & AI output parsing	`src/lib/validation/`
Vitest	3.2.7	Automated unit & integration testing	`tests/` directory
Lucide React	0.475.0	Icons for travel & admin UX	Shared UI components
docx	9.2.0	Programmatic DOCX document generation	`scripts/` & documentation

8. System Architecture

TripCanvas follows a layered architecture separating public presentation, administrative CMS operations, server-side business intelligence, and relational persistence:

```
graph TD
  User[Public Traveller] -->|Browses Catalog / Submits Form| FE[Next.js App Router (SSR & RSC)]
  Staff[Admin / Editor / Consultant] -->|Authenticates via Cookie| Portal[Staff Operations Dashboard

  subgraph Next.js Application Core
    FE --> Router[Route Handlers & Server Actions]
    Portal --> Router
    Router --> Auth[Auth & RBAC Middleware]
    Router --> Val[Zod Schema Validation & Rate Limiting]
    Router --> PricingEngine[Deterministic Pricing Calculator]
    Router --> RecEngine[Package Scoring Matcher]
  end

  subgraph AI Services Layer
    Router -->|Server-Side API Call| Gemini[Google Gemini API (@google/genai)]
    Gemini -->|Structured JSON Extraction| Router
  end

  subgraph Persistence & Audit
    Router --> Store[Unified Data Access Layer (Supabase / In-Memory Store)]
    Store --> DB[(Supabase PostgreSQL)]
    Store --> Audit[(activity_logs Table)]
  end
```

9. Application Modules

- **Catalog Module (`src/app/(public)/packages`):** Renders published travel listings, handles client-side filtering, and renders detail pages with day-wise itineraries, route maps, and seasonal price tables.
- **Enquiry Module (`src/app/(public)/enquire`):** Handles public travel requests with client/server Zod validation, honeypot traps, and automated generation of acknowledgement records.
- **Package CMS Module (`src/app/admin/packages`):** Implements the 10-section package editor, draft persistence, draft preview rendering, transactional publishing, and historical rollback.
- **CRM Inbox Module (`src/app/admin/enquiries`):** Provides staff with centralized enquiry triage, status progression, consultant assignment, and internal note keeping.
- **AI Operational Module (`src/lib/ai/`):** Encapsulates Gemini extraction with strict timeouts, deterministic package recommendation scoring, and editorial quote drafting.
- **Auth & RBAC Module (`src/lib/auth/`):** Enforces role-based permissions across Admin, Editor, Consultant, and Traveller roles using HTTPOnly cookie sessions.
- **Audit Logging Module (`src/app/admin/activity`):** Tracks operational events across all entities to maintain compliance and traceability.
- **Redirects Module (`src/app/admin/redirects`):** Maintains permanent 301 redirects to ensure seamless SEO transition from legacy URLs.

10. Database Design

The relational database schema is structured into 11 tables in PostgreSQL:

```
erDiagram
  profiles ||--o{ package_versions : "creates"
  profiles ||--o{ enquiries : "assigned_to"
  profiles ||--o{ enquiry_notes : "authors"
  profiles ||--o{ enquiry_messages : "approves"
  profiles ||--o{ activity_logs : "triggers"

  packages ||--o{ package_versions : "has_many"
  packages ||--o{ enquiries : "referenced_in"
  packages ||--o{ package_suggestions : "suggested_in"

  package_versions ||--o{ package_version_prices : "contains"
  package_versions ||--o{ package_suggestions : "matched_version"

  enquiries ||--o| ai_extractions : "generates"
  enquiries ||--o{ package_suggestions : "receives"
  enquiries ||--o{ enquiry_notes : "contains"
  enquiries ||--o{ enquiry_messages : "contains"
```

Table Name	Primary Key	Key Foreign Keys	Description & Constraints
profiles	id (UUID)	auth.users(id)	Staff and user profiles with role check constraint ('admin', 'editor', 'consultant', 'traveller').
packages	id (TEXT)	published_version_id, draft_version_id	Master packages table holding unique slug and version pointers.
package_versions	id (UUID)	package_id (packages.id)	Immutable version records storing structured content, status ('draft','published','archived'), and version number.
package_version_prices	id (UUID)	package_version_id (package_versions.id)	Seasonal pricing tiers per version (season_name, start/end month, price_per_person, currency).
enquiries	id (UUID)	package_id, assigned_consultant_id	Traveller enquiries with unique enquiry_number, traveller counts, budget, consent flag, and status.
ai_extractions	id (UUID)	enquiry_id (enquiries.id)	Structured AI extraction output, confidence score, review_needed flag, and model metadata.
package_suggestions	id (UUID)	enquiry_id, package_id, package_version_id	Deterministic recommendation matches with 0-100 score and match reason arrays.
enquiry_notes	id (UUID)	enquiry_id, author_id (profiles.id)	Internal consultant discussion and operational notes.
enquiry_messages	id (UUID)	enquiry_id, approved_by (profiles.id)	Outgoing messages and AI proposal drafts with approval timestamps.
activity_logs	id (UUID)	actor_id (profiles.id)	Audit log entries recording action, entity_type, entity_id, and metadata JSON.
legacy_redirects	id (UUID)	None	Mapping of legacy source paths to new target paths with 301 status code.

11. API Documentation

Method	Endpoint	Auth	Request / Purpose	Success Response
GET	/api/auth/session	Public	Inspect active cookie session	`{ user, isAuthenticated: boolean }`
POST	/api/auth/session	Public	Body: { email } to authenticate demo staff role	`{ success: true, user: Profile }`
DELETE	/api/auth/session	Staff	Clears session cookie	`{ success: true }`
GET	/api/packages	Public / Staff	Query: ?scope=admin to retrieve drafts; default returns published	`{ packages: PackageRecord[] }`
POST	/api/packages	Admin / Editor	Body: PackageEditorInput to create package with v1 draft	201 Created: `{ package: PackageRecord }`
GET	/api/packages/[id]	Public / Staff	Retrieve package by ID with versions	`{ package: PackageRecord }`
PUT	/api/packages/[id]	Admin / Editor	Body: PackageEditorInput to update draft version	`{ success: true, package: PackageRecord }`
POST	/api/packages/[id]/publish	Admin / Editor	Finalizes draft as published; updates pointer	`{ success: true, message: string }`
POST	/api/packages/[id]/restore	Admin Only	Body: { versionNumber } to rollback to historical snapshot	`{ success: true, package: PackageRecord }`
POST	/api/packages/[id]/duplicate	Admin / Editor	Duplicates package into a new draft	`{ success: true, package: PackageRecord }`
GET	/api/enquiries	Staff	Query: ?status=&destination= to filter CRM inbox	`{ enquiries: EnquiryRecord[] }`

POST	/api/enquiries	Public	Body: EnquiryFormInput (with honeypot & rate limit)	201 Created: `{ enquiry: EnquiryRecord }`
GET	/api/enquiries/[id]	Staff	Retrieve enquiry with AI extraction, suggestions, notes	`{ enquiry, aiExtraction, suggestions, notes, messages }`
PATCH	/api/enquiries/[id]	Staff	Body: { status, assignedConsultantId, aiExtractionUpdate }	`{ success: true, ...updatedData }`
POST	/api/enquiries/[id]/notes	Staff	Body: { note } to append internal note	201 Created: `{ success: true, note: EnquiryNote }`
POST	/api/enquiries/[id]/draft-reply	Staff	Body: { packageId } to trigger AI reply generation	`{ success: true, message: EnquiryMessage }`
POST	/api/enquiries/[id]/approve-reply	Consultant / Admin	Body: { messageId } to approve and simulate dispatch	`{ success: true, message: EnquiryMessage }`
GET	/api/activity	Staff	Retrieve audit activity log events	`{ logs: ActivityLogRecord[] }`
GET	/api/redirects	Public / Staff	Retrieve list of active 301 redirects	`{ redirects: LegacyRedirectRecord[] }`
POST	/api/redirects	Admin / Editor	Body: JSON { sourcePath, targetPath } or CSV text	`{ success: true, redirect: LegacyRedirectRecord }`

12. User Workflows

- **Traveller Discovery & Booking:** 1. Lands on editorial homepage -> 2. Explores curated packages (`/packages`) with duration/price/season filters -> 3. Views full package details (`/packages/[slug]`) reviewing twin-sharing rates, GST disclosures, and itinerary -> 4. Clicks 'Enquire Now' -> 5. Completes validated form with consent -> 6. Receives reference number and instant acknowledgement.
- **Content Authoring & Publishing:** 1. Editor signs into `/login` -> 2. Opens Package CMS (`/admin/packages`) -> 3. Edits itinerary or adds seasonal pricing -> 4. Saves draft -> 5. Reviews

draft appearance at `/admin/packages/[id]/preview` -> 6. Administrator executes 'Publish Live' -> 7. Published version becomes active and public.

- **Enquiry Processing & AI Assistance:** 1. Consultant views `/admin/enquiries` -> 2. Opens enquiry details -> 3. Inspects original message and Gemini extracted fields -> 4. Corrects any ambiguous fields -> 5. Reviews deterministic package matches -> 6. Triggers 'Generate Editorial Draft' -> 7. Edits draft and explicitly approves proposal -> 8. Status transitions to 'Replied'.

13. Frontend Design

The frontend architecture utilizes React Server Components (RSC) to maximize performance and SEO, complemented by focused client components for interactivity (e.g. `ItineraryAccordion`, `EnquiryForm`, `PackageEditorForm`). State management utilizes React hooks and URL search params for bookmarkable filters. Forms enforce client-side Zod validation with real-time error feedback, disabled submission states, and accessible error alerts.

14. Backend Design

The backend layer is structured into modular domain services located in `src/lib/`: `db/store.ts` manages transactional persistence; `ai/gemini.ts` manages Google Gen AI interactions; `ai/recommendation.ts` encapsulates the deterministic scoring algorithm; `pricing/calculator.ts` calculates transparent tariffs; and `auth/session.ts` governs cryptographic cookie parsing. All mutations log structured entries into `activity_logs`.

15. Security Considerations

- **Server-Side Secret Isolation:** The Google Gemini API key and Supabase credentials are strictly inaccessible from client runtimes. No secret keys are committed to Git.
- **Input Sanitization & Validation:** All incoming payloads are strictly validated against Zod schemas; malformed or unverified fields are rejected before reaching business logic.
- **Bot Mitigation & Rate Limiting:** Enquiry endpoints feature hidden honeypot fields that trap automated scrapers and in-memory IP rate limiting (10 requests per minute).
- **Row Level Security (RLS):** PostgreSQL tables enforce policies restricting public queries to published records and safeguarding enquiry details to authenticated staff.
- **DPDP Act 2023 Compliance:** Explicit consent tracking for WhatsApp and email communication is maintained with transparent withdrawal notices.

16. Testing Strategy

TripCanvas incorporates an automated test suite executed via Vitest, verifying 15 critical unit and integration test assertions across 5 core suites:

- **Pricing Suite (`tests/unit/pricing.test.ts`):** Verifies correct seasonal selection across Peak (Dec) and Monsoon (Jul) months, strict 5% GST computation, and child disclaimer enforcement.
- **Validation Suite (`tests/unit/validation.test.ts`):** Verifies that invalid email formats and honeypot triggers are rejected, while valid payloads are safely committed.
- **AI & Recommendation Suite (`tests/unit/ai-recommendation.test.ts`):** Verifies that recommendations only reference published packages and that AI fallback handles offline scenarios without dropping enquiries.
- **Versioning Suite (`tests/unit/versioning.test.ts`):** Verifies that draft modifications do not pollute published live content, publishing updates pointers, and rollback creates new version snapshots.
- **Authorization Suite (`tests/unit/authorization.test.ts`):** Verifies that public users cannot access admin operations, editors cannot manage roles, and consultants have scoped access.

Result: All 15 tests have been executed and verified in the workspace environment, passing with zero failures.

17. Deployment Architecture

TripCanvas is configured for automated continuous deployment on Vercel paired with a Supabase PostgreSQL backend. Next.js statically prerenders public marketing routes (`/about`, `/contact`, `/privacy`, `/terms`, `sitemap.xml`) while dynamically executing server routes (`/packages`, `/admin/*`, `/api/*`). Complete migration scripts are maintained in `supabase/migrations/` and detailed deployment steps are provided in `deployment.md`.

18. Environment Variables

Variable Name	Required	Purpose	Safe Placeholder Example
NEXT_PUBLIC_SUPABASE_URL	Yes	Supabase PostgreSQL endpoint	https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Yes	Supabase anonymous public key	sb_publishable_placeholder
NEXT_PUBLIC_SITE_URL	Yes	Canonical URL for SEO & sitemaps	http://localhost:3000
GEMINI_API_KEY	Yes (for AI)	Google Gen AI server-side API key	your-gemini-api-key-here

19. Error Handling and Logging

TripCanvas applies structured error handling across both client and server boundaries. API route handlers wrap operations in try-catch blocks returning standard JSON error responses with appropriate HTTP status codes (400, 401, 403, 404, 429, 500). If Gemini encounters network timeouts or rate limits, the intake pipeline safely logs the warning and falls back to rule-based defaults. Operational changes are recorded in the `activity_logs` table for auditability.

20. Performance and Scalability

Performance optimizations include: Incremental Static Regeneration (ISR) and static page generation for public marketing routes; Next.js Image optimization for remote Unsplash photography; CSS bundle minification via Tailwind; and lightweight Lucide SVG icons. For future enterprise scaling, database connection pooling via Supabase pgBouncer and distributed caching via Redis are recommended.

21. Known Limitations

- **Simulated External Delivery:** Outbound emails and WhatsApp dispatches are simulated as message records in the local database rather than delivered to live telecom carriers.
- **Offline AI Rule Fallback:** When running without a valid Gemini API key, AI extraction relies on regex heuristics and flags the intake for manual consultant review.

- **Single-Region In-Memory State:** The in-memory data store operates on a single Node.js process; production deployment across multiple serverless regions requires executing the provided Supabase SQL migrations.

22. Future Enhancements

- **PDF Proposal Builder:** Automated client-side or server-side generation of branded PDF itinerary brochures with dynamic customer names and prices.
- **Two-Way WhatsApp Webhook Sync:** Live bi-directional synchronization of WhatsApp chat transcripts into the enquiry CRM thread.
- **Hotel Inventory Integrations:** Direct API integration with Channel Managers (e.g. SiteMinder) for real-time room availability confirmation.

23. Installation and Execution

To execute TripCanvas locally:

```
# 1. Install dependencies
npm install

# 2. Execute automated test suite
npm test

# 3. Start development server
npm run dev

# 4. Production build
npm run build && npm run start
```

Refer to `deployment.md` for complete cloud migration and Supabase setup instructions.

25. Conclusion

TripCanvas represents a robust, production-quality solution tailored to the operational and brand requirements of Company A, Kochi. By combining an editorial design aesthetic with versioned CMS immutability, deterministic pricing controls, server-side AI extraction, and role-based access control, the platform resolves developer bottlenecks and transforms travel enquiry triage into an efficient, professional workflow.